Clinic_Management_System

Clinic Appointment & Patient Management System Report

DATABASE SYSTEMS SUMMATIVE PROJECT

Group B(Clinic Appointment & Patient Management System)

Team Members:

Halimatu Sadia Mohammed -
Hanif Olayiwola -

Part I - Database Design & Normalisation

Task I.1 - Entities, Attributes and Un-Normalised Form (UNF)

Before creating the relational database, all entities and their attributes were identified exactly as they appeared in the original Clinic Appointment and Patient Management System developed in Programming 1. At this stage, no normalisation has been applied. The purpose of the UNF stage is to identify repeating groups, derived attributes, and candidate keys that may later require restructuring.

a. Patients (UNF)

patient_id	name	age	contact	gender
P001	Kingston Adeniyi	21	55186584	Male

Primary Key Candidate: patient_id

No repeating groups or derived attributes exist in this entity.

b. Doctors (UNF)

doctor_id	name	specialty	available_days	start_time	end_time

D001	Dr. Amina Mensah	General Consultation	Mon, Wed, Fri	08:00	16:00

Primary Key Candidate: doctor_id
Repeating Group: available_days

The available_days column contains multiple values in a single field, which violates First Normal Form.

c. Appointments (UNF)

appointment_id	patient_id	doctor_id	date	time	duration	department	purpose	status	end_time
A001	P001	D001	2026-10-02	10:30	30	General Consultation	Annual Checkup	Booked	11:00

Primary Key Candidate: appointment_id
Derived Attribute: end_time

d. Departments (UNF)

department_id	department_name	description
DEPT01	General Consultation	Routine checkups and GP visits

Primary Key Candidate: department_id
Candidate Key: department_name

Summary of Issues Identified in UNF

Entity	Issue Identified
Doctors	available_days contains multiple values (Repeating Group)
Appointments	end_time is a derived attribute
Appointments	department stored as text may create a transitive dependency

Task I.2 - Normalisation to Third Normal Form (3NF)

Step 1 - First Normal Form (1NF)

Rule: (Each column must contain a single atomic value, and repeating groups must be removed.)

a. Violated Functional Dependency

doctor_id → available_days
The available_days attribute contains multiple values for a single doctor.

b. Transformation Justification

The repeating group was removed from the Doctors table and placed into a new relation called Doctor Availability.

c. Resulting Tables:

Doctors (1NF)

doctor_id	name	specialty	start_time	end_time
D001	Dr. Amina Mensah	General Consultation	08:00	16:00

DoctorAvailability (1NF)

doctor_id	available_day
D001	Mon
D001	Wed
D001	Fri

Primary Key: (doctor_id, available_day)
Foreign Key: doctor_id → Doctors(doctor_id)

d. Justification

Each row now contains only one day value, ensuring atomicity and compliance with 1NF.
All other entities (Patients, Appointments, and Departments) were already in 1NF because every cell already held a single atomic value.

Step 2 - Second Normal Form (2NF)

Rule**:** Must already be in 1NF. Every non-key attribute must depend on the entire primary key.

The only table with a composite primary key(PK) is DoctorAvailability (doctor_id + available_day).

Functional Dependency Analysis: (doctor_id, available_day) → No non-key attributes
It has no other columns, so there is nothing that could be partially dependent on just one part of the key.

Transformation Justification

Since all other tables have a single-column P, and no non-key attributes depend on only part of the composite key, the table already satisfies 2NF.

Step 3 – Third Normal Form (3NF)

Rule**:** Must already be in 2NF. Then, every non-key column must depend directly on the primary key, not on another non-key column. This is called a transitive dependency.

a. Violated Functional Dependency

In the Appointments table, the department column stores a text name like 'General Consultation'.
If we wanted to store a description or extra info about the department, it would depend on department name, not on appointment_id.
That creates a transitive dependency: appointment_id → department_name → description.

b. Transformation Applied

We replace the department text column with a department_id foreign key(FK). All department details are stored in the Departments table, where department_id is the primary key.

c. Resulting Tables after department text column is replaced with department_id FK:

Appointments (3NF)

appointment_id	patient_id	doctor_id	department_id	date	time	duration	purpose	status
A001	P001	D001	DEPT01	2026-10-02	10:30	30	Annual Checkup	Booked

The Departments table (which was already planned) now formally holds all department information:

Departments (3NF)

department_id	department_name	description
DEPT01	General Consultation	Routine checkups and GP visits

Derived Attribute Removed

end_time was removed because:
end_time = time + duration
The value can always be calculated when needed.

Justification

All other tables (Patients, Doctors, and DoctorAvailability)have no transitive dependencies and therefore pass 3NF without any changes. Since department information is stored in one location, it helps to eliminate redundancy and ensure every non-key attribute depends directly on the primary key.

Final 3NF Schema

After going through 1NF, 2NF, and 3NF, the database now has 6 tables:

Table	Primary Key	Foreign Keys	Change from UNF
Patients	patient_id	No Foreign Key	No change
Doctors	doctor_id	No Foreign Key	available_days column removed
DoctorAvailability	(doctor_id, available_day)	doctor_id → Doctors	NEW table (1NF fix)
Departments	department_id	No Foreign Key	No change
Appointments	appointment_id	patient_id, doctor_id, department_id	department text replaced with FK

The database is now fully normalised to Third Normal Form (3NF), with repeating groups removed, partial dependencies eliminated, and transitive dependencies resolved.

Task I.3 – Constraints Documentation

The following tables document all constraints for each relation in the final 3NF schema, covering primary keys, candidate/unique keys, domain constraints (data types, NOT NULL, CHECK), default values, foreign keys, and referential integrity actions.

Relation Name: Patients

Column Name	Data Type	PK / FK / UQ	NOT NULL?	CHECK Constraint	DEFAULT	FK References / RI Action
patient_id	VARCHAR(10)	PK	YES	–	–	–
name	VARCHAR(100)	–	YES	–	–	–
age	INT	–	YES	age >= 1 AND age <= 120	–	–
contact	VARCHAR(20)	UQ	YES	–	–	–
gender	VARCHAR(10)	–	YES	gender IN ('Male', 'Female', 'Other')	–	–

Relation Name: Doctors

Column Name	Data Type	PK / FK / UQ	NOT NULL?	CHECK Constraint	DEFAULT	FK References / RI Action
doctor_id	VARCHAR(10)	PK	YES	–	–	–
name	VARCHAR(100)	–	YES	–	–	–
specialty	VARCHAR(100)	–	YES	–	–	–
start_time	TIME	–	YES	–	–	–
end_time	TIME	–	YES	end_time > start_time	–	–

Relation Name: DoctorAvailability

Column Name	Data Type	PK / FK / UQ	NOT NULL?	CHECK Constraint	DEFAULT	FK References / RI Action
doctor_id	VARCHAR(10)	PK, FK	YES	–	–	Doctors(doctor_id) ON DELETE CASCADE ON UPDATE CASCADE
available_day	VARCHAR(10)	PK	YES	available_day IN ('Mon','Tue','Wed','Thu','Fri','Sat','Sun')	–	–

Relation Name: Departments

Column Name	Data Type	PK / FK / UQ	NOT NULL?	CHECK Constraint	DEFAULT	FK References / RI Action
department_id	VARCHAR(10)	PK	YES	–	–	–
department_name	VARCHAR(100)	UQ	YES	–	–	–
description	TEXT	–	NO	–	NULL	–

Relation Name: Appointments

Column	Data Type	PK/FK /UQ	NOT NULL?	CHECK Constraint	DEFAULT	FK Reference / RI Action
appointment_id	VARCHAR(10)	PK	YES	–	–	–
patient_id	VARCHAR(10)	FK	YES	–	–	Patients(patient_id) ON DELETE RESTRICT ON UPDATE CASCADE
doctor_id	VARCHAR(10)	FK	YES	–	–	Doctors(doctor_id) ON DELETE RESTRICT ON UPDATE CASCADE
department_id	VARCHAR(10)	FK	YES	–	–	Departments(department_id) ON DELETE RESTRICT ON UPDATE CASCADE

date	DATE	–	YES	–	–	–
time	TIME	–	YES	–	–	–
duration	INT	–	YES	duration > 0	–	–
purpose	VARCHAR(255)	–	YES	–	–	–
status	VARCHAR(15)	–	YES	status IN ('Booked', 'Cancelled')	'Booked'	–

Part II - SQL Implementation

Task II.1 - DDL: Schema, Relations, and Constraints

Write CREATE statements to build the full database schema, including:

- CREATE DATABASE / USE / CREATE SCHEMA statements.
- CREATE TABLE statements for every relation.
- All PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT constraints are defined inline or via ALTER TABLE.
- Correct ON DELETE / ON UPDATE referential actions (RESTRICT, CASCADE, SET NULL, etc.).

Link:
https://github.com/Sadia-ALCHE/Clinic_Management_Database/blob/main/Clinic_Management_ddl.sql

Task II.2 - DML: Data Insertion

- Populate every table with realistic sample data:
- A minimum of 5 rows per table is required.
- Data must be consistent across related tables (i.e., foreign key values must exist in the referenced table).M
- Include at least one deliberate attempt to violate a referential integrity constraint and document the database's response (error message/behaviour) in your written report.

Link:
https://github.com/Sadia-ALCHE/Clinic_Management_Database/blob/main/Clinic_Management_dml.sql

Referential Integrity Constraint Test

To test referential integrity, an attempt was made to insert an appointment record using a patient ID that does not exist in the `Patient` table.

SQL Statement:

```
-- DELIBERATE REFERENTIAL INTEGRITY VIOLATION --------------

-- Uncomment to test:
/*
INSERT INTO Appointment
    (appointment_id, patient_id, doctor_id, department_id, date, time, duration, purpose, status)
VALUES
('A008', 'P999', 'D001', 'DEPT02', '2026-06-15', '09:00:00', 30, 'Invalid Patient Test', 'Booked');
*/
```

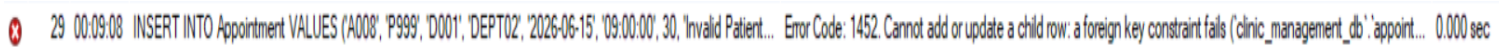
INSERT INTO Appointment
(appointment_id, patient_id, doctor_id, department_id, date, time, duration, purpose, status)
VALUES
('A008', 'P999', 'D001', 'DEPT02', '2026-06-15', '09:00:00', 30, 'Invalid Patient Test', 'Booked');

Result:

MySQL returned a foreign key constraint error because the value P999 does not exist in the Patient table.

Example Error Message:

Cannot add or update a child row: a foreign key constraint fails (`clinic_management_db`.`appointment`, CONSTRAINT `fk_appt_patient` FOREIGN KEY (`patient_id`) REFERENCES `patient` (`patient_id`) ON DELETE RESTRICT ON UPDATE CASCADE)



0	29	00:09:08	INSERT INTO Appointment VALUES ('A008', 'P999', 'D001', 'DEPT02', '2026-06-15', '09:00:00', 30, 'Invalid Patient Test', 'Booked')	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (`clinic_management_db`.`appointment`, CONSTRAINT `fk_appt_patient` FOREIGN KEY (`patient_id`) REFERENCES `patient` (`patient_id`) ON DELETE RESTRICT ON UPDATE CASCADE)	0.000 sec

Explanation:

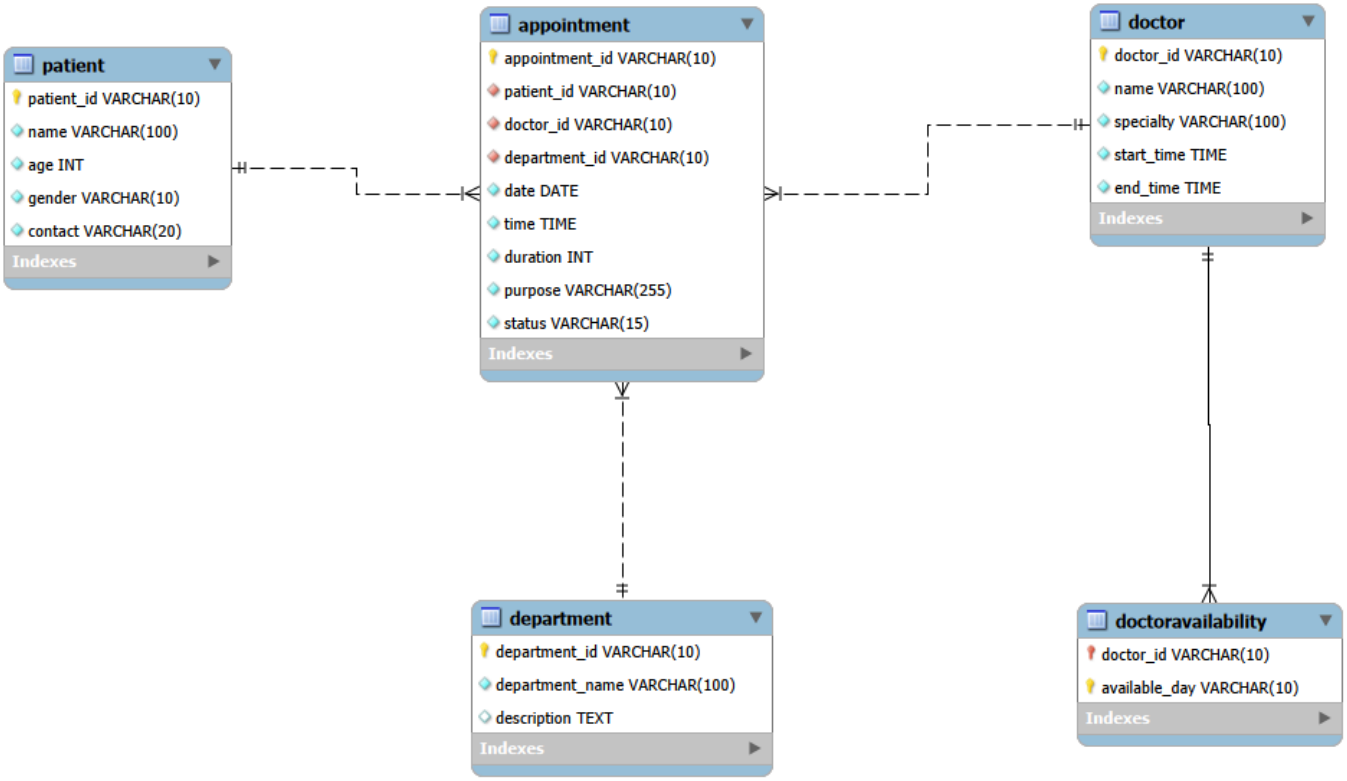
This demonstrates that referential integrity constraints are correctly enforced in the database. The patient_id column in the Appointment table is linked to the patient_id column in the Patient table through a foreign key relationship. Therefore, appointments cannot be created for patients that do not exist in the system.

Part III - Entity-Relationship Diagram

Produce a complete, professional ERD/EER that accurately represents the final 3NF database. The ERD/EER must include:

- All entities and their attributes (primary keys underlined or marked with PK).
- All relationships with correct cardinality notation (crow's foot or UML notation).
- Participation constraints (mandatory / optional).
- Foreign key references clearly shown.

The ERD may be produced using any diagramming tool (MySQL Workbench, Lucidchart, etc.) and must be submitted as a high-resolution image or PDF embedded within your written report. A hand-drawn ERD will not be accepted.



Entity Relationship Diagram (ERD)

The ERD below represents the final 3NF structure of the Clinic Appointment & Patient Management System database.
The diagram shows:

- All entities and their attributes
- Primary keys and foreign keys
- Relationships between tables
- Cardinality constraints between entities

The database was designed and normalised to Third Normal Form (3NF) to eliminate redundancy and ensure data integrity.

GitHub Repository

GitHub Repository

The complete SQL scripts, database backup file, and supporting project materials for this project are available at:

https://github.com/Sadia-ALCHE/Clinic_Management_Database